

Social Computing

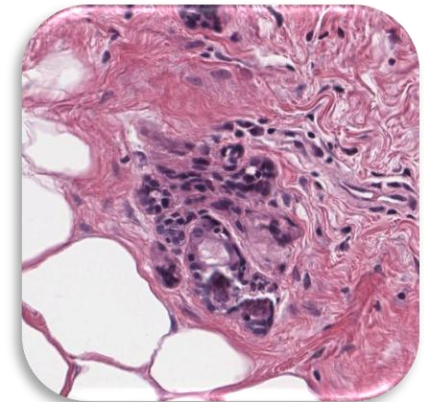
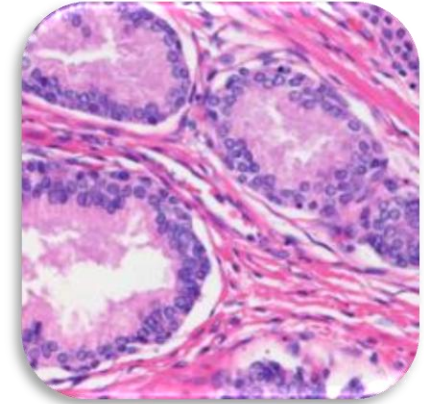
16. NetworkX

16.a Graphs with NetworkX

Hafsa AKEBLI
Date: 24/11/2025

About Me

- *Second-year PhD Student in Computer Science and Artificial Intelligence*
- **PhD Topic:** *Deep Learning for Disease Pattern Detection in Pathology*
- **Objective:** *Use AI to assist pathologists during the diagnosis of different types of cancer*



Histopathological Patches



Introduction : Graphs

NetworkX

- First Steps
- Creating a graph
- Nodes
- Edges
- Removing elements from a graph
- Examining graphs
- Attributes of graph/node/edge
- Directed graphs
- Graph Generators and Graph Operations

Graphs

A graph **G** is type of data structure written as **G = (V, E)** where:

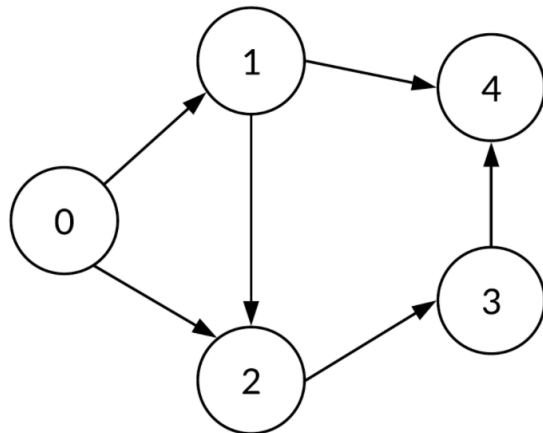
- **V** is the set of nodes/vertices
- **E** is the set of edges connecting pairs of nodes

Graphs can be **Directed or Undirected**



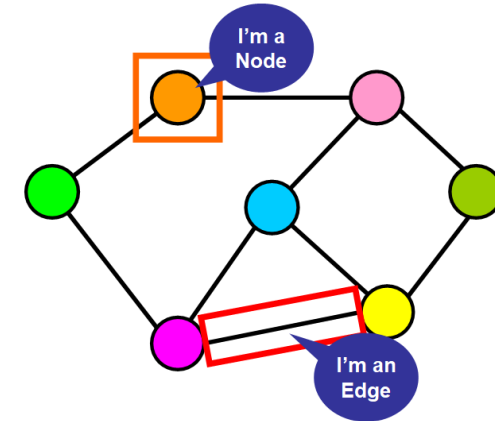
A is the adjacency matrix of the graph; a square matrix used to describe the connections between the nodes where:

- $A_{ij} = 1$ if there is an edge from **node i to node j**
- $A_{ij} = 0$ Otherwise

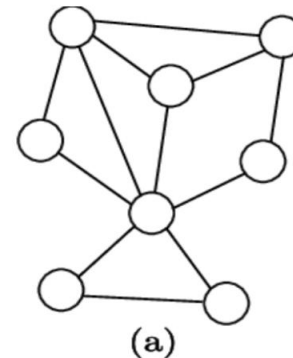


Adjacency Matrix

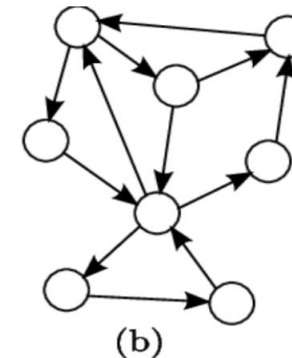
	0	1	2	3	4
0	0	1	1	0	0
1	0	0	1	0	1
2	0	0	0	1	0
3	0	0	0	0	1
4	0	0	0	0	0



Undirected graph

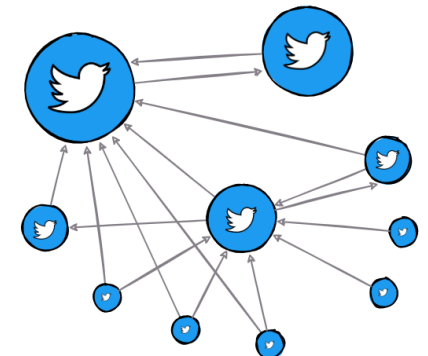


Directed graph

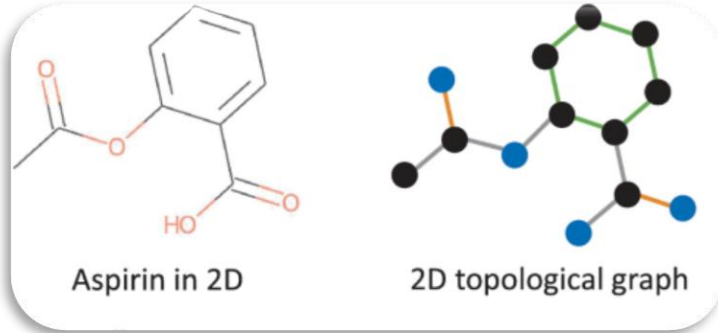


Social Networks as Graphs

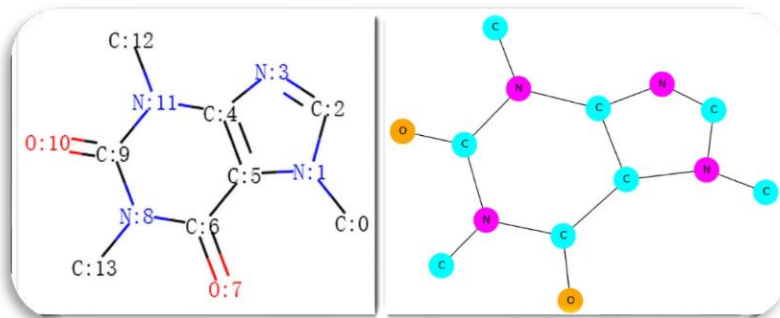
- **Social Networks** can be represented as graphs : **Nodes represent persons**, and **Edges relationships between them** (friends, colleagues, family...)
- **Facebook** can be represented with a **undirected graph** (e.g: Alice and Bob are friends, and vice versa)
- **Twitter** can be represented with a **directed graph** (e.g: Alice is following Bob, but it doesn't implicate Bob is following Alice)



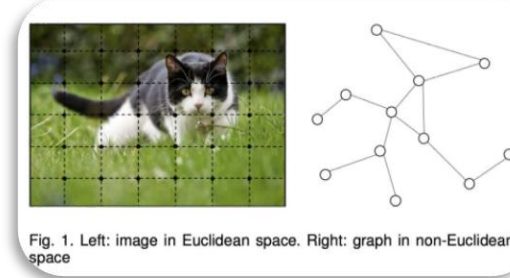
Graphs in Real-World Applications



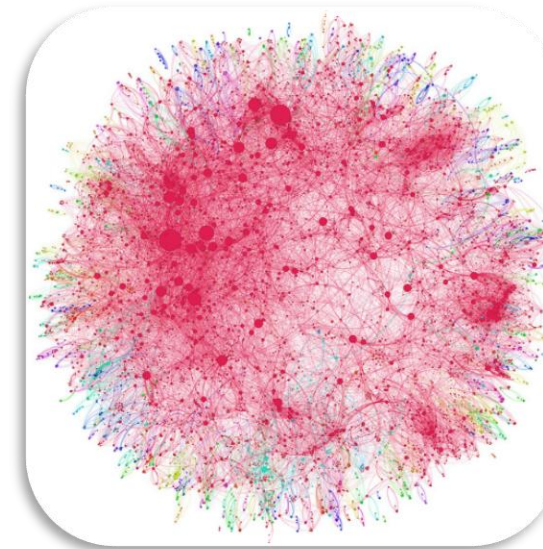
Bioinformatics & Drug Discovery:
Molecular structures, protein interactions



Chemistry:
Molecules as graphs (e.g. **Caffeine**:
nodes = atoms, edges = bonds)



Computer Vision: Image
representation, object detection



Academia: Co-authorship network map of
physicians publishing on hepatitis C, Nodes
(doctors), edges(collaborations)

Graphs in Medical Imaging

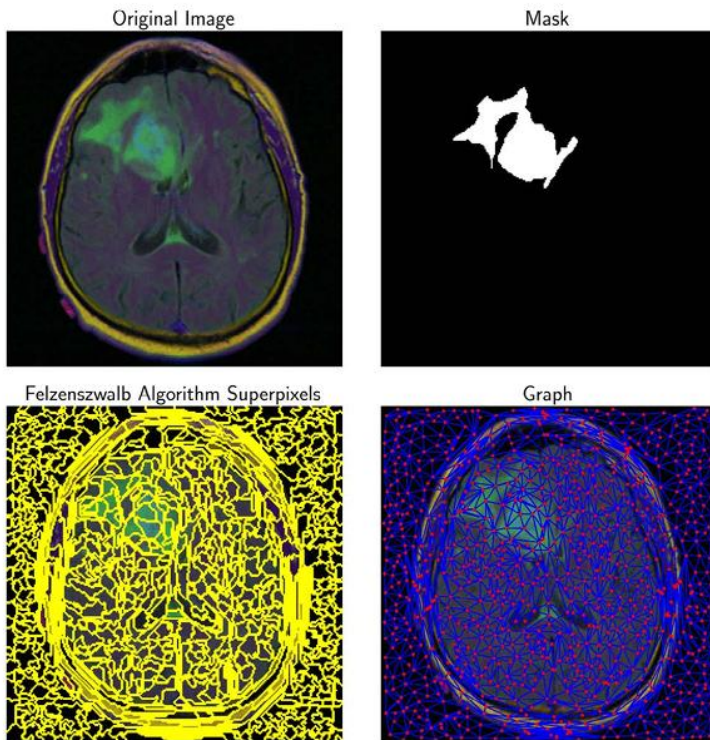
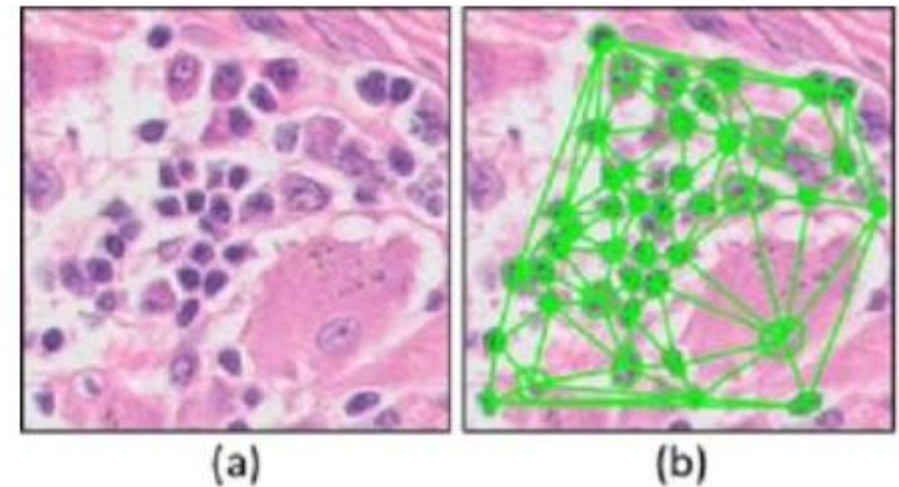


Fig. 2: Graph building procedure.

Radiology:

Brain MRI as a graph to detect Gliomas(tumors); Nodes=Superpixels

Automated glioma detection and segmentation using graphical models; DOI: [10.1371/journal.pone.0200745](https://doi.org/10.1371/journal.pone.0200745)



Pathology:

Cell Graph, Nodes=Nuclei, Predict Survival in Early-Stage Lung Cancer

Feature Driven Local Cell Graph (FeDeG): Predicting Overall Survival in Early Stage Lung Cancer; DOI: [10.1007/978-3-030-00934-2_46](https://doi.org/10.1007/978-3-030-00934-2_46)



NetworkX



NetworkX
Network Analysis in Python

<https://networkx.org/>

- **Python library for creating, analyzing, and visualizing graphs**

Offers:

- Data structures, algorithms, network structure and analysis measures for graphs
- **Nodes** can be «anything» (e.g: text, images)
- **Edges** can hold arbitrary data (e.g: weights, time-series)



Code For This Part

The code for this part (tutorial) is available on Teams

Open the notebook : “**networkx_Corso.ipynb**”

Location: 16.NetworkX.zip → 16.a.Graphs with NetworkX

Getting Started with NetworkX

- Installation of NetworkX:

conda install -c anaconda networkx

pip install networkx

- To **update** the library: **pip install --upgrade networkx**
- We will import as well some libraries that we will need:
 - **NumPy** the fundamental package for scientific computing with python
 - **Pandas** a library used for data analysis and dataset manipulation

```
import os
import networkx as nx
import pandas as pd
import numpy as np
```

✓ 0.2s



NetworkX – Creating a graph

- Nodes can be any immutable object : **an object having a unique identifier that doesn't change**, e.g.: an integer, a text string, a tuple, an image, an XML object, another Graph, a customized node object ... "Vincenzo", 17, (17,"Roma") ..
- Mutable data structures like lists, dictionaries.. Can't be nodes of a graph
- Example of graph creation : To represent a Twitter user :
 - We can consider **the user's ID as a node**



NetworkX – Adding Nodes

- **networkx.Graph()** to construct an empty graph with no nodes and no edges

Different ways to add nodes:

- **add_node(n)** to add node n , add a node once a time
- **add_nodes_from(list)** to add nodes from any iterable container like a list
- **add_nodes_from(graph)** Also nodes from one graph can be incorporated into another graph

```
graph = nx.Graph()
graph_second = nx.Graph()

graph.add_node("Wednesday")
graph.add_nodes_from(["Social", "Computing", 2020])

graph_second.add_nodes_from(range(0,10))

graph.add_nodes_from(graph_second)
print(graph.nodes)
```

i8] ✓ 0.0s P₃

• ['Wednesday', 'Social', 'Computing', 2020, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

NetworkX – Adding Edges

- **graph.add_edge(n1, n2)** Add one edge between nodes n1 and n2
- **graph.add_edges_from(list)** Add multiple edges from a list
- **graph.add_edges_from(graph1.edges)** adds all edges from «graph1» in «graph»
- **graph.clear()** Remove all nodes and edges from the graph

```
graph_second.add_edge(4,5)
graph.add_edges_from([(0, 1), (0, 3), (1, 2), (2, 3)])
graph.add_edges_from(graph_second.edges)
print(graph.edges)
```

[45] ✓ 0.0s

... [(0, 1), (0, 3), (1, 2), (2, 3), (4, 5)]



NetworkX – Exploring the Graph

We can examine the content of our graph using:

- **`graph.nodes()`** , **`graph.edges()`** : View all nodes and edges of the graph
- **`graph.number_of_nodes()`** and **`graph.number_of_edges()`** to show the total number of nodes and edges
- **`graph.adj[n]`** : View the adjacent nodes (neighbors) of node **n**
- **`graph.degree[n]`** to view the degree of node **n**: **number of edges connected to node n**

```
print(graph.nodes())  
print(graph.edges())  
print(graph.number_of_nodes())  
print(graph.number_of_edges())  
print(graph.adj[0])  
print(graph.degree[0])
```

```
graph.degree()
```

```
[47]
```

```
✓ 0.0s
```

```
Python
```

```
... DegreeView({'Wednesday': 0, 'Social': 0, 'Computing': 0, 2020: 0, 0: 2, 1: 2, 2:
```

```
[46]
```

```
✓ 0.0s
```

```
... ['Wednesday', 'Social', 'Computing', 2020, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[(0, 1), (0, 3), (1, 2), (2, 3), (4, 5)]
```

```
14
```

```
5
```

```
{1: {}, 3: {}}
```

```
2
```

NetworkX – Exploring the Graph

- **graph[1]** is the same as **graph.adj[1]** : Access the adjacency information for **node 1**
- **graph[1][2]** is the same as **graph.edges[1,2]** : used to access the attributes between node **1** and **2**.
- To set the attribute of the edge (1,2):

graph[1][2]["color"] = "yellow" or
graph.edges[1,2] ["color"] = "yellow"

```
> print(graph[1], "-----", graph.adj[1])
> print(graph[1][2], "-----", graph.edges[1, 2])
> graph[1][2]["color"] = "yellow"
> print(graph[1][2], "-----", graph.edges[1, 2])

[6] ✓ 0.0s

... {0: {}, 2: {}} ----- {0: {}, 2: {}}
    {} ----- {}
    {'color': 'yellow'} ----- {'color': 'yellow'}
```

NetworkX – Exploring the Graph

- **graph.adjacency()** or **graph.adj.items()** to view all *(node, adjacency)* pairs

```
▶ ▾  
for node, adj in graph.adjacency():  
    print(node, adj)  
[7] ✓ 0.0s  
... Wednesday {}  
    Social {}  
    Computing {}  
    2020 {}  
    0 {1: {}, 3: {}}  
    1 {0: {}, 2: {'color': 'yellow'}}  
    2 {1: {'color': 'yellow'}, 3: {}}  
    3 {0: {}, 2: {}}  
    4 {5: {}}  
    5 {4: {}}  
    6 {}  
    7 {}  
    8 {}  
    9 {}
```

OR: We can get only the pairs of nodes that are connected by edges using graph.edges()

```
▶ ▾  
for node_1, node_2 in graph.edges():  
    print(node_1, node_2)  
[8] ✓ 0.0s  
... 0 1  
    0 3  
    1 2  
    2 3  
    4 5
```

NetworkX – Density of a Graph

- **Density:** The density of a graph measures how many edges are in the graph compared to the maximum number of possible edges.
- For undirected graphs the density of a *graph*(n, m); (n : number of nodes, m : number of edges) is :

$$d = \frac{2m}{n(n-1)},$$

- We multiply by 2 because every edge is counted twice
- For directed graphs the density of a *graph*(n, m) is:

$$d = \frac{m}{n(n-1)},$$

```
density_graph = nx.density(graph)
print (density_graph)
```

[11] ✓ 0.0s

... 0.05128205128205128

NetworkX – Removing Nodes and Edges

- We can also remove nodes and edges from the graph in a similar way to adding
- **`graph.remove_node(n)`** to remove node `n`, **`graph.remove_nodes_from(list)`**,
- **`graph.remove_edge(n1,n2)`** , **`graph.remove_edges_from(list)`** to remove a list of edges at once



```
graph.remove_node("Wednesday")
graph.remove_edge(4,5)
print(graph.nodes())
print(graph.edges())
print(graph.number_of_nodes())
print(graph.number_of_edges())
```

[9]

✓ 0.0s

```
... ['Social', 'Computing', 2020, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
    [(0, 1), (0, 3), (1, 2), (2, 3)]
    13
    4
```

NetworkX – Graph Attributes

- Attributes like weights, labels, colors, or whatever Python object you want, can be attached to graphs, nodes, or edges
- By default, the attributes are empty dictionaries
- **Graph Attributes of a graph G:**

```
▶ G = nx.Graph(day="Tuesday")  
  G.graph  
[10] ✓ 0.0s  
... {'day': 'Tuesday'}
```

- You can change this Graph attribute:

```
▶ G.graph['day'] = "Monday"  
  G.graph  
[11] ✓ 0.0s  
... {'day': 'Monday'}
```

NetworkX – Node and Edge Attributes

Node and Edges attributes:

- Attributes can be added to nodes using: **graph.add_node()**, **graph.add_nodes_from()**
- Attributes can be added to edges using : **graph.add_edge()** , **graph.add_edges_from()**
- To get all nodes and edges with their attributes we can use respectively:

graph.nodes().data(), graph.edges().data()

```
print(G.nodes().data())
print(G.edges().data())
print(G.nodes["374"])
print(G.edges["374", "11"]["status"])
```

[14] ✓ 0.0s

```
... [('374', {'username': 'JohnSmith', 'kind': 'Employer'}), ('451', {'kind': 'Employee'}), ('11', {'kind': 'Employee'})]
[('374', '11', {'status': 'Leader'}), ('374', '451', {'status': 'Leader'}), ('451', '11', {'status': 'Colleague'})]
{'username': 'JohnSmith', 'kind': 'Employer'}
Leader
```

```
G.add_node("374", username="JohnSmith", kind="Employer")
G.add_nodes_from(["451", "11"], kind='Employee')
G.add_edge("374", "11", status="Leader")
G.add_edge("374", "451", status="Leader")
G.add_edges_from([("11", "451", {"status": "Colleague"})])
```

✓ 0.0s

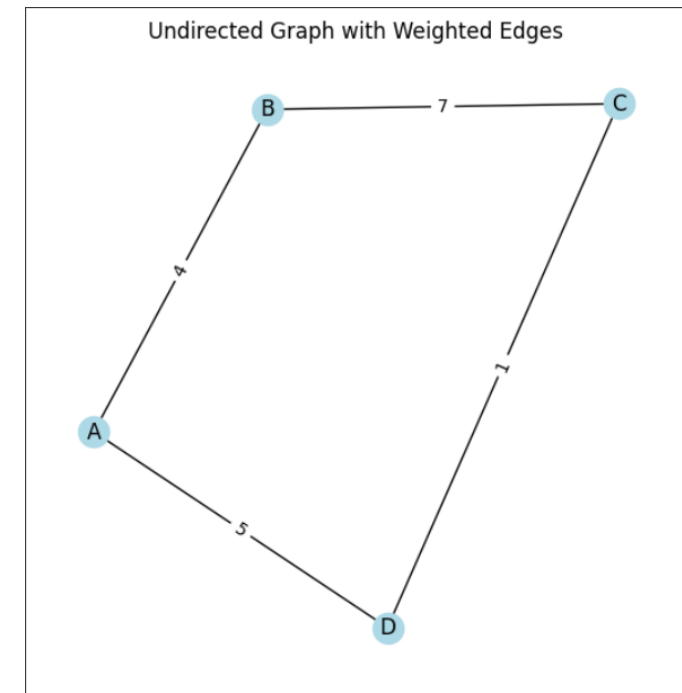
NetworkX – Weighted Edges

- In weighted graphs, edges have an associated value “**weight**” that can represent qualitative or quantitative properties between nodes, for **e.g.** in social networks they can represent the **number of interactions** between users (messages, likes, comments..)
- Use the function **draw_networkx_edge_labels()** to show the labels in the graph representation (we will see later)



```
#weighted undirected graph
weighted_undirected_graph = nx.Graph()
# Add edges with weights
weighted_undirected_graph.add_edge("A", "B", weight=4)
weighted_undirected_graph.add_edge("B", "C", weight=7)
weighted_undirected_graph.add_edge("C", "D", weight=1)
weighted_undirected_graph.add_edge("D", "A", weight=5)
```

[15] ✓ 0.0s

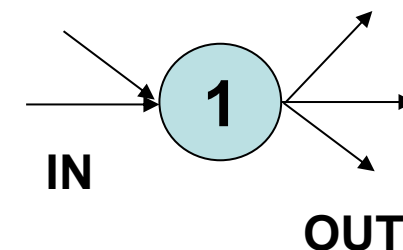


NetworkX – Directed Graphs

- The **DiGraph class** provides methods and properties specific to directed edges
- The directed versions of «**neighbors**» is equivalent to «**successors**»
- So a unique code can work with both directed and undirected graphs
- **DiGraph.degree** reports the sum of **DiGraph.in_degree** and **DiGraph.out_degree**

```
directed_graph = nx.DiGraph()
directed_graph.add_weighted_edges_from([(1, 2, 0.5), (3, 1, 0.75)])
directed_graph.add_node(5)
```

.8] ✓ 0.0s



```
directed_graph.out_degree(1, weight='weight')
```

0.5

```
directed_graph.in_degree(1, weight='weight')
```

[]
... 0.75

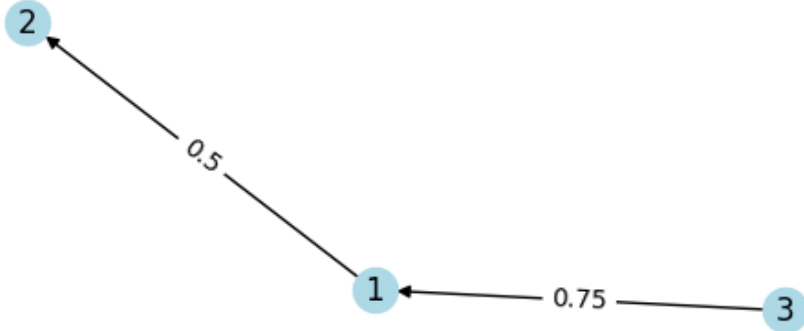
```
directed_graph.degree(1, weight='weight')
```

[]
... 1.25

NetworkX – Directed Graphs

directed_graph with Weighted Edges

5



Without specifying the weight parameter:

The function **counts the number of edges** connected to or from the node.

```
directed_graph.degree(1)
```

```
2
```

```
directed_graph.in_degree(1)
```

```
1
```

```
directed_graph.out_degree(1)
```

```
1
```

NetworkX – Directed Graphs

- View the nodes (no attributes here), edges with their weights:

```
▶ ▼  
    print(directed_graph.nodes().data())  
    print(directed_graph.edges().data())  
[20] ✓ 0.0s  
... [(1, {}), (2, {}), (3, {}), (5, {})]  
    [(1, 2, {'weight': 0.5}), (3, 1, {'weight': 0.75})]
```

- Neighbors equivalent to «**successors**» of graph

```
▶ ▼  
    print(list(directed_graph.neighbors(1)), "--", list(directed_graph.successors(1)))  
[21] ✓ 0.0s  
... [2] -- [2]
```

- «**Predecessors**» of a node: Nodes that have a directed edge pointing toward that node

```
▶ ▼  
    list(directed_graph.predecessors(2))  
[22] ✓ 0.0s  
... [1]
```



NetworkX – Directed Graphs

Convert a directed graph to an
undirected graph:

graph.undirected()

```
[23] H=directed_graph.to_undirected()  
✓ 0.0s
```

```
[24] H.nodes().data()  
✓ 0.0s  
... NodeDataView({1: {}, 2: {}, 3: {}, 5: {}})
```

```
[25] H.edges().data()  
✓ 0.0s  
... EdgeDataView([(1, 2, {'weight': 0.5}), (1, 3, {'weight': 0.75})])
```

NetworkX – Directed Graphs

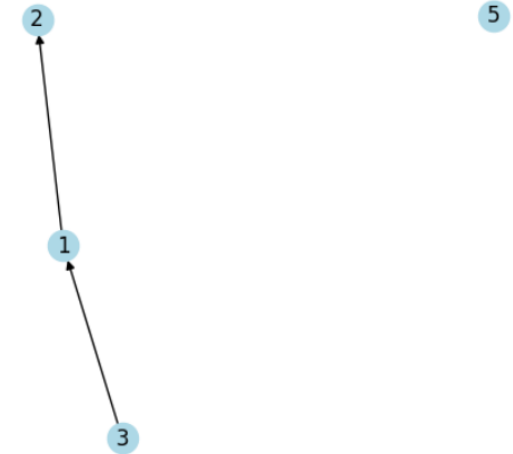


```
#let's draw this indirected graph  
plt.figure(figsize=(4, 4))  
nx.draw(directed_graph, with_labels=True, node_color='lightblue')  
plt.title("directed_graph")  
plt.show()
```

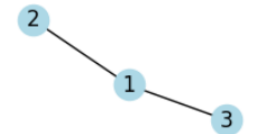


```
plt.figure(figsize=(4, 4))  
nx.draw(H, with_labels=True, node_color='lightblue')  
plt.title("undirected_graph")  
plt.show()
```

directed_graph



undirected_graph



No more
arrows



- So far we have constructed graphs node-by-node or edge-by-edge
- Graphs can also be generated using different methods :
 - Applying **Classic Graph Operations** (cartesian product, union..)
 - Using **Predefined functions** to generate «Classic» graphs
 - Using a **Constructive graph generator** for a classic graph: based on specific rules or patterns.
 - Using a **Stochastic graph generator**
 - **Load saved graphs** from standard format like adjacency lists, pickle format, GML, GraphML...



Applying classic graph operations - Examples

<code>subgraph</code> (G, nbunch)	Returns the subgraph induced on nodes in nbunch.
<code>union</code> (G, H[, rename])	Combine graphs G and H.
<code>disjoint_union</code> (G, H)	Combine graphs G and H.
<code>cartesian_product</code> (G, H)	Returns the Cartesian product of G and H.
<code>compose</code> (G, H)	Compose graph G with H by combining nodes and edges into a single graph.
<code>complement</code> (G)	Returns the graph complement of G.
<code>create_empty_copy</code> (G[, with_data])	Returns a copy of the graph G with all of the edges removed.
<code>to_undirected</code> (graph)	Returns an undirected view of the graph <code>graph</code> .
<code>to_directed</code> (graph)	Returns a directed view of the graph <code>graph</code> .

Applying classic graph operations - examples

Union: Combines graphs **G** and **H**. The names of nodes must be unique. A renaming facility is provided to avoid name collisions.

```
#union of H and G
H = nx.Graph()
H.add_edges_from([(1, 2), (2, 3)])

G = nx.Graph()
G.add_edges_from([(3, 4), (4, 5)])

union_graph = nx.union(H, G, rename=('H', 'G'))
plt.figure(figsize=(4, 4))
nx.draw(union_graph, with_labels=True, node_color='lightblue')
plt.title("Union of H and G")
plt.show()
```

Union of H and G



Applying classic graph operations - examples

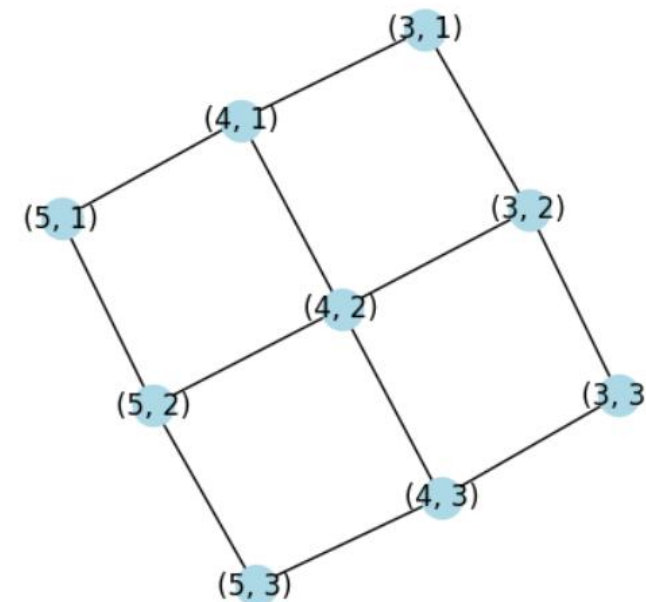
- The Cartesian Product P of the graphs G and H has a node set that is the Cartesian Product of the node sets, $V(P) = V(G) \times V(H)$,
- P has an edge $((u,v), (x,y))$ if and only if $u = x$ and v and y are adjacent in H , or if $v = y$ and u and x are adjacent in G .



```
#cartesian product
P = nx.cartesian_product(G, H)
plt.figure(figsize=(4, 4))
nx.draw(P, with_labels=True, node_color='lightblue')
plt.title("Cartesian Product of H & G")
plt.show()
```

[31]

Cartesian Product of H & G



NetworkX – Graph generators and graph operations

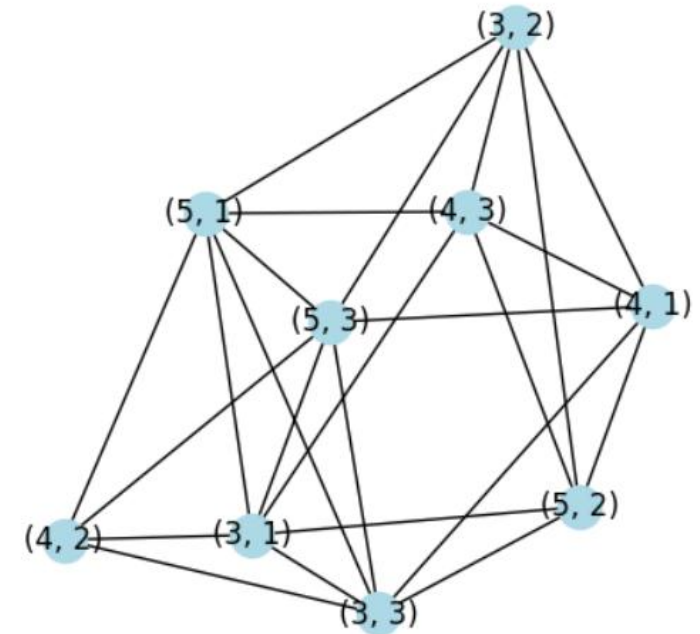
Applying classic graph operations - examples

Graph complement of a Graph

The **complement of a graph** G is a graph G' that has the same set of nodes as G , but two nodes in G' are connected by an edge if and only if they are **not connected** by an edge in G

```
#complement
G_complement = nx.complement(P)
plt.figure(figsize=(4, 4))
nx.draw(G_complement, with_labels=True, node_color='lightblue')
plt.title("Complement of the graph P, the cartesian product of G and H :)")
plt.show()
```

Complement of the graph P, the cartesian product of G and H :)





NetworkX – Graph generators and graph operations

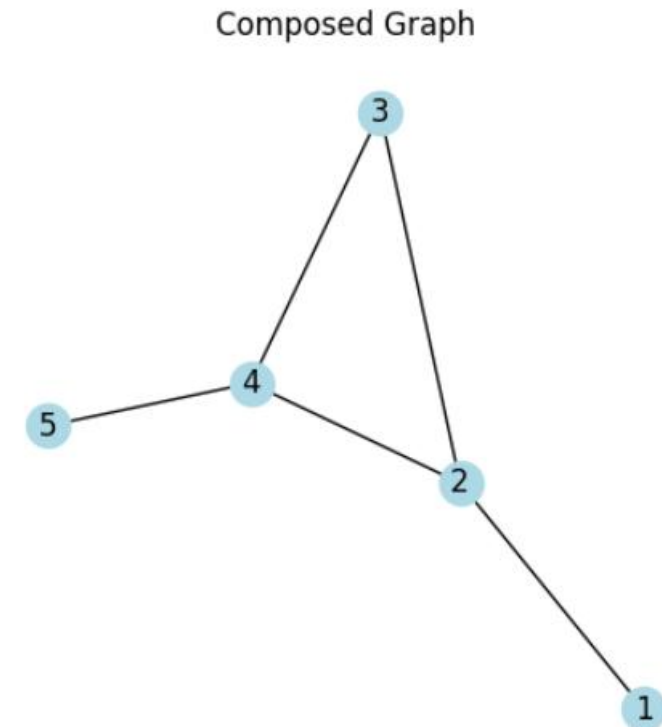
Applying classic graph operations - examples

Compose 2 Graphs:

Compose graph G1 with G2 by combining nodes and edges into a single graph.

```
# Compose 2 Graphs
G1 = nx.Graph()
G1.add_edges_from([(1, 2), (2, 3), (2, 4)])
G2 = nx.Graph()
G2.add_edges_from([(3, 4), (4, 5)])
# Compose G1 and G2
composed_graph = nx.compose(G1, G2)
plt.figure(figsize=(4, 4))
nx.draw(composed_graph, with_labels=True, node_color='lightblue')
plt.title("Composed Graph")
plt.show()
```

[8] ✓ 0.0s



Using a call to one of the classic small graphs:

Predefined classic graphs

`petersen_graph` ([create_using])

Returns the Petersen graph.

`tutte_graph` ([create_using])

Returns the Tutte graph.

`sedgewick_maze_graph` ([create_using])

Return a small maze with a cycle.

`tetrahedral_graph` ([create_using])

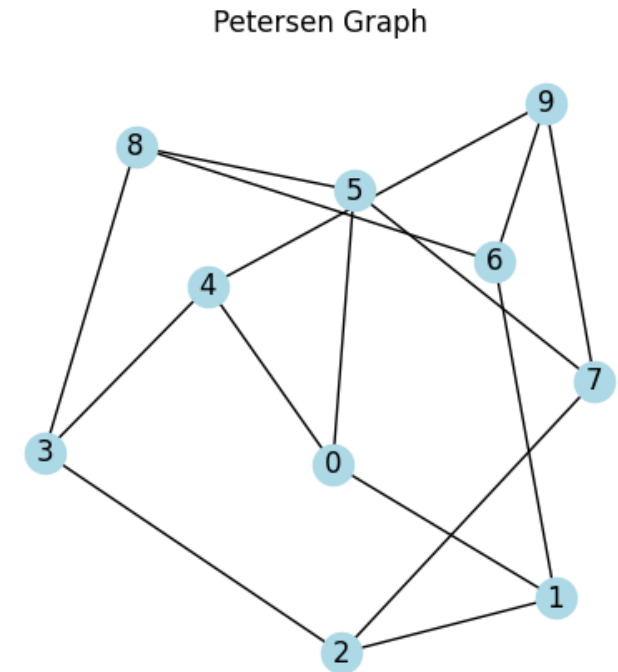
Returns the 3-regular Platonic Tetrahedral graph.

Using a call to one of the classic small graphs: Predefined classic graphs

Petersen Graph:

- An undirected graph with 10 nodes and 15 edges.

```
# Create Petersen Graph
petersen_graph = nx.petersen_graph()
plt.figure(figsize=(4, 4))
nx.draw(petersen_graph, with_labels=True, node_color='lightblue')
plt.title("Petersen Graph")
plt.show()
```



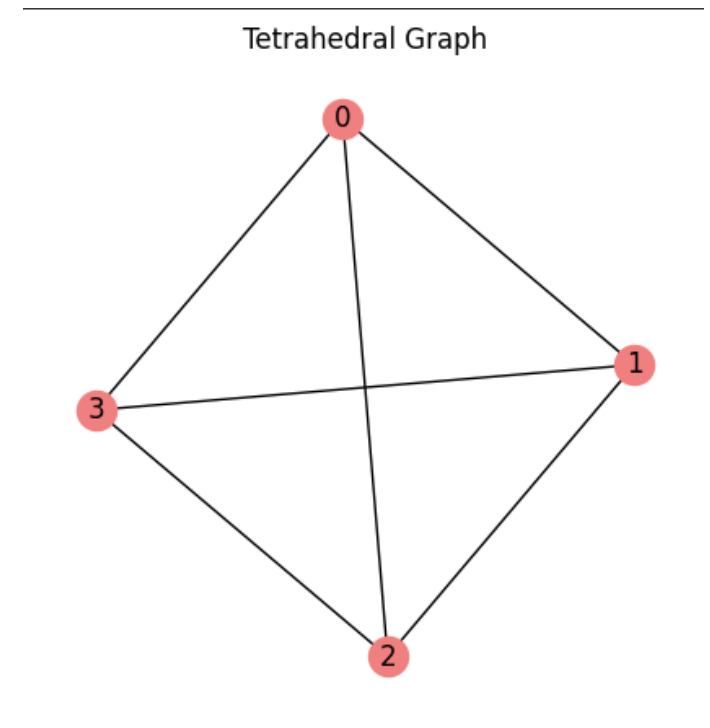
Using a call to one of the classic small graphs: Predefined classic graphs

Tetrahedral Graph:

Tetrahedral graph has 4 nodes and 6 edges.

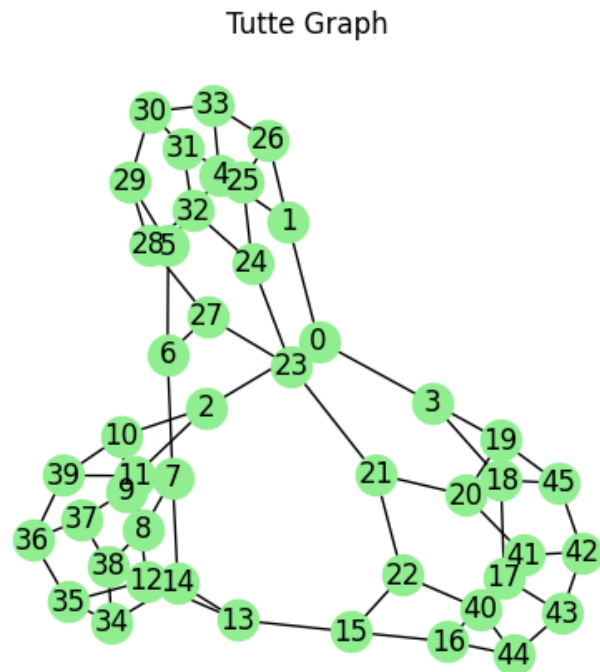
```
# Create Tetrahedral Graph
tetrahedral_graph = nx.tetrahedral_graph()
# Draw the graph
plt.figure(figsize=(4, 4))
nx.draw(tetrahedral_graph, with_labels=True, node_color='lightcoral')
plt.title("Tetrahedral Graph")
plt.show()
```

Pj

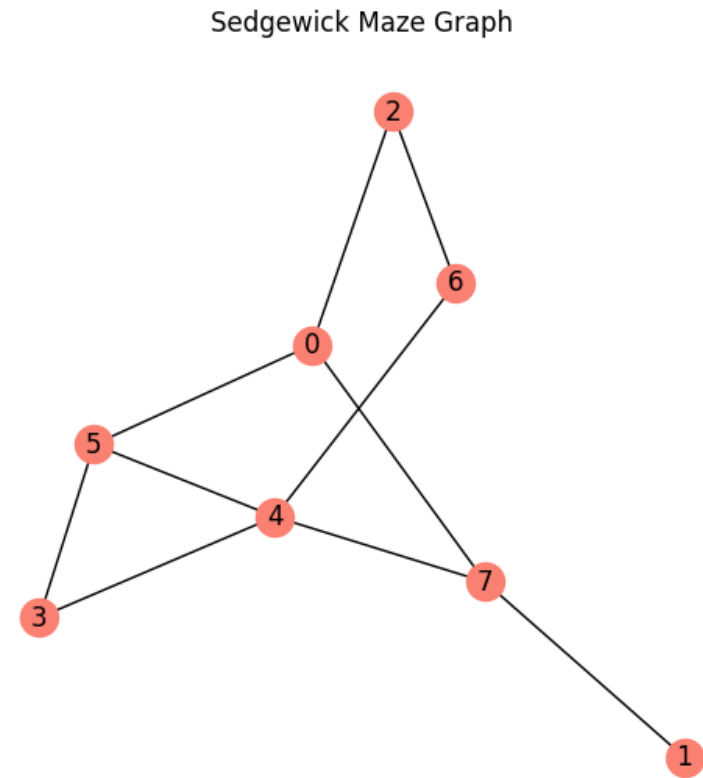


Using a call to one of the classic small graphs:
Predefined classic graphs

Tutte Graph



Sedgewick Maze Graph



Using a constructive generator for a classic graph:

Predefined graph generators

`complete_graph`(n[, create_using])

Return the complete graph K_n with n nodes.

`complete_bipartite_graph`(n1, n2[, create_using])

Returns the complete bipartite graph $K_{\{n_1, n_2\}}$.

`barbell_graph`(m1, m2[, create_using])

Returns the Barbell Graph: two complete graphs connected by a path.

`lollipop_graph`(m, n[, create_using])

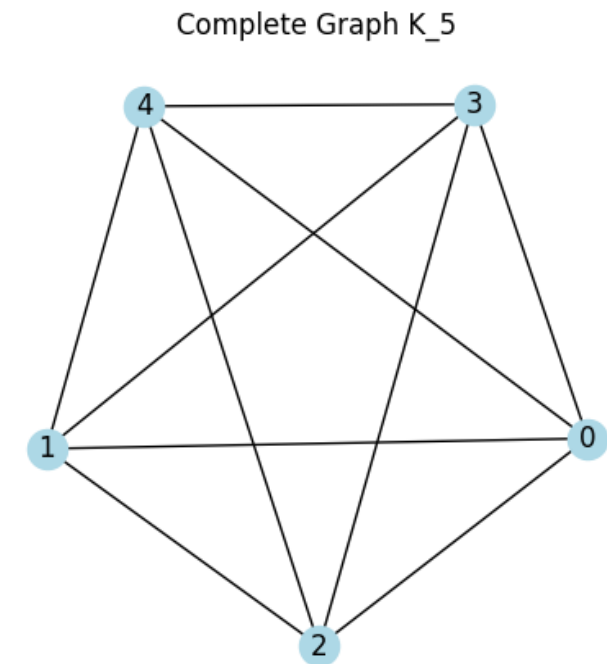
Returns the Lollipop Graph; K_m connected to P_n .

Using a constructive generator for a classic graph

Complete Graph:

- A graph where every pair of nodes is connected by an edge.
- K_n : Represents a complete graph with n nodes.

```
● # Create a complete graph with 5 nodes  
complete_graph = nx.complete_graph(5)  
  
# Draw the graph  
plt.figure(figsize=(4, 4))  
nx.draw(complete_graph, with_labels=True, node_color='lightblue')  
plt.title("Complete Graph K_5")  
plt.show()
```



Using a constructive generator for a classic graph

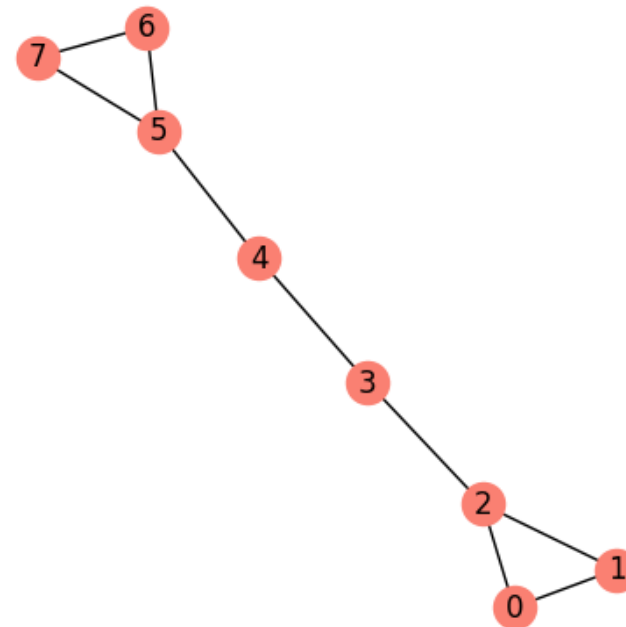
Barbell Graph:

- Two complete graphs connected by a path.
- **barbell_graph(m1, m2)** where **m1**: Size of the left and right barbells, **m2**: Length of the path connecting the barbells.

```
# Create a Barbell Graph with K_3 connected by a path of length 2
barbell_graph = nx.barbell_graph(3, 2)

# Draw the graph
plt.figure(figsize=(4, 4))
nx.draw(barbell_graph, with_labels=True, node_color='salmon')
plt.title("Barbell Graph (K_3 connected by a path of length 2)")
plt.show()
```

Barbell Graph (K_3 connected by a path of length 2)



Using a stochastic graph generator

Stochastic graph generators create graphs based on probabilistic models

`erdos_renyi_graph`(n, p[, seed, directed, ...])

Returns a $G_{n,p}$ random graph, also known as an Erdős-Rényi graph or a binomial graph.

`watts_strogatz_graph`(n, k, p[, seed, ...])

Returns a Watts–Strogatz small-world graph.

`barabasi_albert_graph`(n, m[, seed, ...])

Returns a random graph using Barabási–Albert preferential attachment

`random_lobster`(n, p1, p2[, seed, create_using])

Returns a random lobster graph.

Using a stochastic graph generator

Erdős-Rényi graph or a Binomial graph:

$G_{n,p}$: a random graph where :

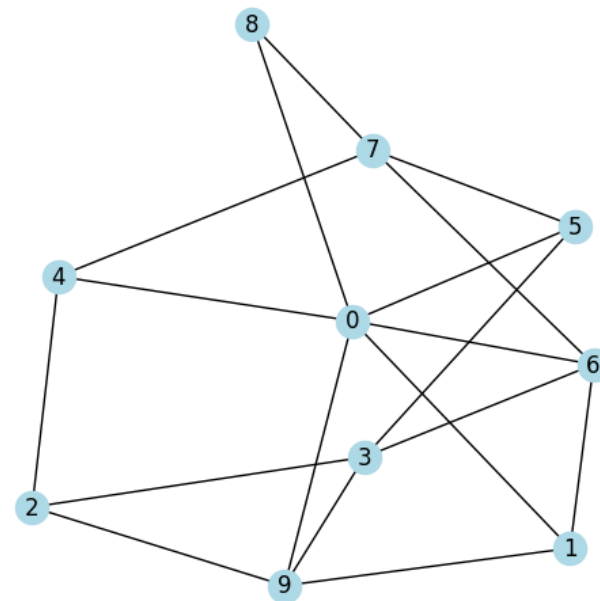
- **n** : the number of nodes
- **p** : probability for edge creation

```
# Create an Erdős-Rényi Graph with 10 nodes and edge probability 0.4
n, p = 10, 0.4
erdos_renyi = nx.erdos_renyi_graph(n, p)

# Draw the graph
plt.figure(figsize=(5, 5))
nx.draw(erdos_renyi, with_labels=True, node_color='lightblue')
plt.title(f"Erdős-Rényi Graph (n={n}, p={p})")
plt.show()
```

F

Erdős-Rényi Graph (n=10, p=0.4)

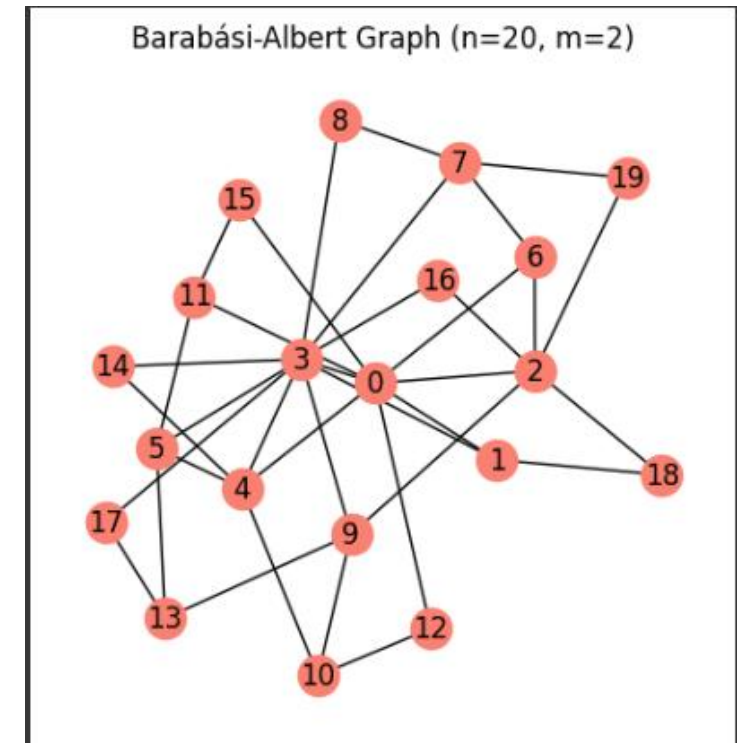


Using a stochastic graph generator

Barabási-Albert Graph :

- A random graph of **n** nodes is grown by attaching new nodes each with **m** edges that are preferentially attached to existing nodes with high degree.

```
# Create a Barabási-Albert Graph with 20 nodes and 2 edges added per step
n, m = 20, 2
barabasi_albert = nx.barabasi_albert_graph(n, m)
plt.figure(figsize=(4, 4))
nx.draw(barabasi_albert, with_labels=True, node_color='salmon')
plt.title(f"Barabási-Albert Graph (n={n}, m={m})")
plt.show()
```





Exercises for practice:

- The exercises are available on the **Teams Group**
- Open the notebook : “**NetworkX_Exercises**”
 - **Location: 16.NetworkX.zip → 16.a.Graphs with NetworkX**
- Try to solve the exercises independently.
- The correction notebook is available in the same folder for self-check: Notebook “**NetworkX_Correction.ipynb**”

Social Computing

16. NetworkX

16.b Game of Graphs

Hafsa AKEBLI
Date: 24/11/2025



- Graph Matrix
- Drawing graphs (graph layout..)
- **Game of Graphs**
 - Dataset Description
 - Data loading
 - Exploring the dataset
 - Graph Creation
 - Save the Graph
 - Load the Graph
 - Drawing of the graph
 - Reduced Graph



Code For This Part

The code for this part (tutorial) is available on Teams

Notebook Name: «game_of_graphs.ipynb»

Location: 16.NetworkX.zip → 16.b.Game of Graphs

Graph Matrix

Adjacency Matrix

The **adjacency matrix** is a way to represent a graph in a matrix form where:

- **Rows and Columns:** Represent nodes of the graph.
- Entries: the value at position $A[i,j]$:
 - **For unweighted graphs:** **1** if there is an edge between **node i** and **node j**, otherwise **0**
 - **For weighted graphs:** the entry corresponds to the weight of the edge

```
# Create a weighted undirected graph
graph = nx.Graph()
graph.add_edge(1, 2, weight=3)
graph.add_edge(1, 3, weight=2)
graph.add_edge(2, 3, weight=5)
graph.add_edge(3, 4, weight=1)
# Compute adjacency matrix
adj_matrix = nx.adjacency_matrix(graph)
# Print the matrix as a dense array for better readability
print("Adjacency Matrix:")
print(adj_matrix.toarray())
```

Adjacency Matrix:

```
[[0 3 2 0]
 [3 0 5 0]
 [2 5 0 1]
 [0 0 1 0]]
```

Graph Matrix

Incidence Matrix

- **Rows** represent **nodes**.
- **Columns** represent **edges**.

Each entry in the matrix indicates the relationship between a node and an edge.

Standard Incidence Matrix (oriented=False):

1 appears wherever a node is connected to an edge

Oriented Incidence Matrix (oriented=True):

- For **directed graphs**, an edge has a **source** and a **destination**: **+1** for the destination, **-1** for the source
- For **undirected graphs**, orientations are assigned arbitrarily.
- **Zeroes** appear for all other entries.

```
>   
• # Compute incidence matrix  
  incidence_matrix = nx.incidence_matrix(graph, oriented=False)  
  # Convert to dense format for better visualization  
  print(incidence_matrix.toarray())
```

[3]

```
... [[1. 1. 0. 0.]  
     [1. 0. 1. 0.]  
     [0. 1. 1. 1.]  
     [0. 0. 0. 1.]
```

```
>   
• # Compute incidence matrix  
  incidence_matrix_oriented = nx.incidence_matrix(graph, oriented=True)  
  # Convert to dense format for better visualization  
  print(incidence_matrix_oriented.toarray())
```

[4]

```
... [[-1. -1.  0.  0.]  
     [ 1.  0. -1.  0.]  
     [ 0.  1.  1. -1.]  
     [ 0.  0.  0.  1.]
```

Draw networks with matplotlib library

<code>draw</code> (G[, pos, ax])	Draw the graph G with Matplotlib.
<code>draw_networkx</code> (G[, pos, arrows, with_labels])	Draw the graph G using Matplotlib.
<code>draw_networkx_nodes</code> (G, pos[, nodelist, ...])	Draw the nodes of the graph G.
<code>draw_networkx_edges</code> (G, pos[, edgelist, ...])	Draw the edges of the graph G.
<code>draw_networkx_labels</code> (G, pos[, labels, ...])	Draw node labels on the graph G.
<code>draw_networkx_edge_labels</code> (G, pos[, ...])	Draw edge labels.
<code>draw_circular</code> (G, **kwargs)	Draw the graph <code>G</code> with a circular layout.
<code>draw_kamada_kawai</code> (G, **kwargs)	Draw the graph <code>G</code> with a Kamada-Kawai force-directed layout.
<code>draw_planar</code> (G, **kwargs)	Draw a planar networkx graph <code>G</code> with planar layout.
<code>draw_random</code> (G, **kwargs)	Draw the graph <code>G</code> with a random layout.
<code>draw_spectral</code> (G, **kwargs)	Draw the graph <code>G</code> with a spectral 2D layout.
<code>draw_spring</code> (G, **kwargs)	Draw the graph <code>G</code> with a spring layout.
<code>draw_shell</code> (G[, nlist])	Draw networkx graph <code>G</code> with shell layout.



<https://matplotlib.org/>

Graph Layout: Algorithm used to determine the position of nodes and edges in a graph for visualization. It defines how the elements of the graph (nodes and edges) are spatially arranged.

<code>arf_layout</code> (G[, pos, scaling, a, etol, dt, ...])	Arf layout for networkx
<code>bipartite_layout</code> (G, nodes[, align, scale, ...])	Position nodes in two straight lines.
<code>bfs_layout</code> (G, start, *, align, scale, center])	Position nodes according to breadth-first search algorithm.
<code>circular_layout</code> (G[, scale, center, dim])	Position nodes on a circle.
<code>forceatlas2_layout</code> (G[, pos, max_iter, ...])	Position nodes using the ForceAtlas2 force-directed layout algorithm.
<code>kamada_kawai_layout</code> (G[, dist, pos, weight, ...])	Position nodes using Kamada-Kawai path-length cost-function.
<code>planar_layout</code> (G[, scale, center, dim])	Position nodes without edge intersections.
<code>random_layout</code> (G[, center, dim, seed])	Position nodes uniformly at random in the unit square.

<code>rescale_layout</code> (pos[, scale])	Returns scaled position array to (-scale, scale) in all axes.
<code>rescale_layout_dict</code> (pos[, scale])	Return a dictionary of scaled positions keyed by node
<code>shell_layout</code> (G[, nlist, rotate, scale, ...])	Position nodes in concentric circles.
<code>spring_layout</code> (G[, k, pos, fixed, ...])	Position nodes using Fruchterman-Reingold force-directed algorithm.
<code>spectral_layout</code> (G[, weight, scale, center, dim])	Position nodes using the eigenvectors of the graph Laplacian.
<code>spiral_layout</code> (G[, scale, center, dim, ...])	Position nodes in a spiral layout.
<code>multipartite_layout</code> (G[, subset_key, align, ...])	Position nodes in layers of straight lines.

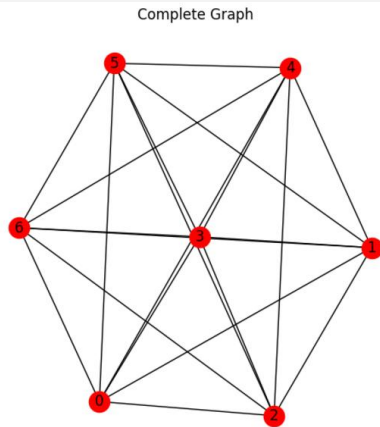
Drawing graphs

Basic draw(graph) function:

- Import libraries
- Create graph
- Create figure with chosen size
- Draw graph, specifying the parameters..color of edges and nodes..

```
• ✓ import networkx as nx  
import matplotlib.pyplot as plt
```

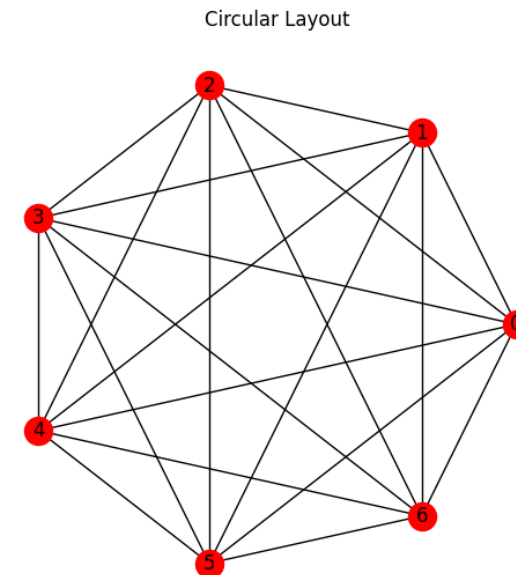
```
G = nx.complete_graph(7)  
plt.figure(figsize=(5, 5))  
nx.draw(G, with_labels=True, node_color="red", edge_color="black")  
plt.title("Complete Graph")  
plt.show()
```



draw_circular(graph) function:

- Draw the graph with a circular layout
- Places nodes equidistantly on a circle.

```
# Circular Layout  
plt.figure(figsize=(5, 5))  
nx.draw_circular(G, with_labels=True, node_color="red", edge_color="black")  
plt.title("Circular Layout")  
plt.show()
```

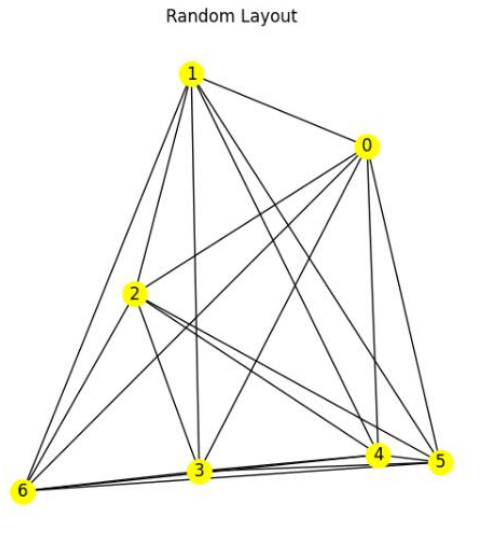


Drawing graphs

Draw_random(graph) function:

- Draw the graph with a random layout

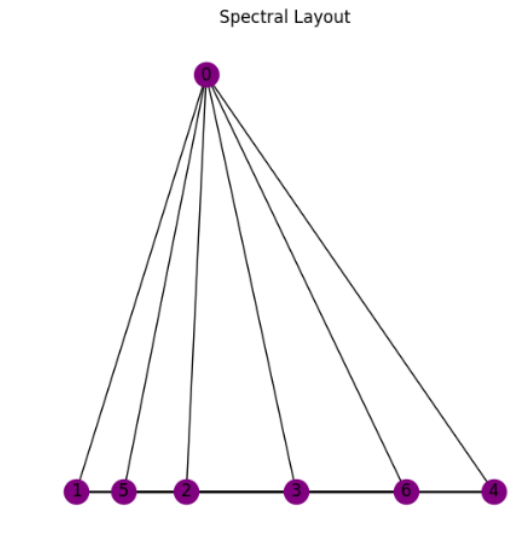
```
# Random Layout
plt.figure(figsize=(5, 5))
nx.draw_random(G, with_labels=True, node_color="yellow", edge_color="black")
plt.title("Random Layout")
plt.show()
```



draw_spectral(graph) function:

- The **spectral layout** arranges the nodes of a graph in 2D space by using the **graph Laplacian**, a mathematical representation of the graph's structure.

```
#spectral layout
plt.figure(figsize=(5, 5))
nx.draw_spectral(G, with_labels=True, node_color="purple", edge_color="black")
plt.title("Spectral Layout")
plt.show()
```



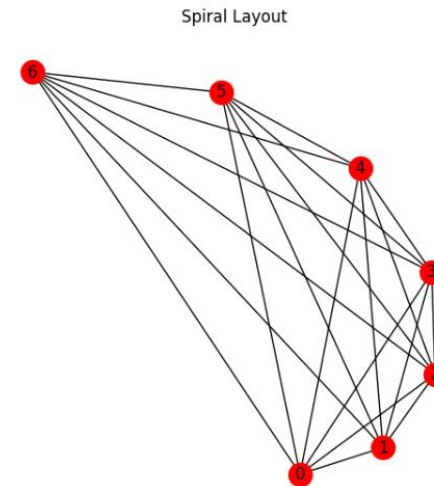
Using Graph Layout

Spiral_layout: Position nodes in a spiral layout

Define the graph layout, then draw the graph with `nx.draw()`, specifying the graph layout in the parameters, `pos` is a dictionary with the positions of the nodes

```
# Spiral Layout
pos = nx.spiral_layout(G)
plt.figure(figsize=(5, 5))
nx.draw(G, pos, with_labels=True, node_color="red", edge_color="black")
plt.title("Spiral Layout")
plt.show()
```

Pyt



```
> print(pos)
```

Python

```
{0: array([-0.03290017, -0.83350958]), 1: array([ 0.26701538, -0.72403186]), 2: array([ 0.45548546, -0.42214804]), 3: array([ 0.44368158, -0.00267732]), 4: array([0.18416294, 0.42499707]), 5: array([-0.317
```

Game of Thrones – Game of Graphs

- Now let's construct undirected graphs representing the interactions between characters from Game of Thrones (the famous book and serie 😊)
- The data is available in this repository:
<https://github.com/mathbeveridge/asoiaf>
- **Interaction Rule:** Two characters are connected by an edge **if their names appear within 15 words of each other** in the book's text.
- We will construct the graphs for all the 5 books
- For each book we have the nodes and edges files to construct the graphs



In this exemple, we will use these files:

- **asoiaf-all-nodes.csv**

Contains information about the characters : their unique **Ids** and **labels(names)**

- Example of a row : **Id**: Aegon-I-Targaryen, **Label**: Aegon I Targaryen

- **asoiaf-all-edges.csv**

Contains information about the connections (edges) between characters, such as:

- **Source** and **target** characters involved in each interaction.
- The **type** of connection (e.g. undirected)
- The **weight** of the edge : the number of interactions between the characters
- Example of a row: **Source**: Aegon-Frey-(son-of-Stevron), **Target**: Catelyn-Stark, **Type**: Undirected, **Id**:12, **weight**:4



Data loading

- The data is stored in csv format, values are separated by commas
- We will use **Pandas library** to load and handle the dataset
- We will load the dataset into **Dataframes**, like table data structures (with labeled rows and columns) that can hold heterogeneous data types (e.g.: numbers, strings...). This makes it easy to manipulate the data.

Data loading

- **Mount Google Drive**, if we are using google colab and the data is stored in Google Drive
- Read the CSV files using Pandas into **Dataframes** : `pd.read_csv("path")`
- **Display the dataframes to see their structure, size..**



```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
nodes_df = pd.read_csv("/content/drive/MyDrive/networks_material/asoiat-all-nodes.csv")
edges_df = pd.read_csv("/content/drive/MyDrive/networks_material/asoiat-all-edges.csv")
display(nodes_df, edges_df)
```

	Id	Label
0	Addam-Marbrand	Addam Marbrand
1	Aegon-Frey-(son-of-Stevron)	Aegon Frey (son of Stevron)
2	Aegon-I-Targaryen	Aegon I Targaryen
3	Aegon-Targaryen-(son-of-Rhaegar)	Aegon Targaryen (son of Rhaegar)
4	Aegon-V-Targaryen	Aegon V Targaryen
...	...	...
791	Yorke-Terys	Yorke Terys
792	Ysilla	Ysilla
793	Yurkhaz-zo-Yunzak	Yurkhaz zo Yunzak
794	Zei	Zei
795	Zollo	Zollo

796 rows x 2 columns

	Source	Target	Type	id	weight
0	Addam-Marbrand	Brynden-Tully	Undirected	0	3
1	Addam-Marbrand	Cersei-Lannister	Undirected	1	3
2	Addam-Marbrand	Gyles-Rosby	Undirected	2	3
3	Addam-Marbrand	Jaime-Lannister	Undirected	3	14
4	Addam-Marbrand	Jalabhar-Xho	Undirected	4	3
...	...	...	...	...	...
2818	Walder-Frey-(son-of-Merrett)	Wex-Pyke	Undirected	2818	4
2819	Waymar-Royce	Will-(prologue)	Undirected	2819	18
2820	Weasel	Weese	Undirected	2820	6
2821	Woth	Yoren	Undirected	2821	4
2822	Yandry	Ysilla	Undirected	2822	14

2823 rows x 5 columns

Exploring the Dataset

dataframe.columns to view all the columns of a dataframe

```
print(nodes_df.columns)
print(edges_df.columns)

Index(['Id', 'Label'], dtype='object')
Index(['Source', 'Target', 'Type', 'id', 'weight'], dtype='object')
```

- **head(n)**: displays the first n rows of the DataFrame
- **tail(n)**: displays the last n rows

```
display(edges_df.head(3))
display(edges_df.tail(3))
```

	Source	Target	Type	id	weight
0	Addam-Marbrand	Brynden-Tully	Undirected	0	3
1	Addam-Marbrand	Cersei-Lannister	Undirected	1	3
2	Addam-Marbrand	Gyles-Rosby	Undirected	2	3

	Source	Target	Type	id	weight
2820	Weasel	Weese	Undirected	2820	6
2821	Woth	Yoren	Undirected	2821	4
2822	Yandry	Ysilla	Undirected	2822	14

- **.loc** : Filter rows based on specific conditions in the dataset.
- In this example, we search for all rows where the «Source» column has a value «Robb-Stark». These rows correspond to interactions where Robb-Stark is the source character
- Then we display the first five rows of the filtered data (**head(5)**)
- And we used (**.values**) to extract the unique names of the column “**Target**”

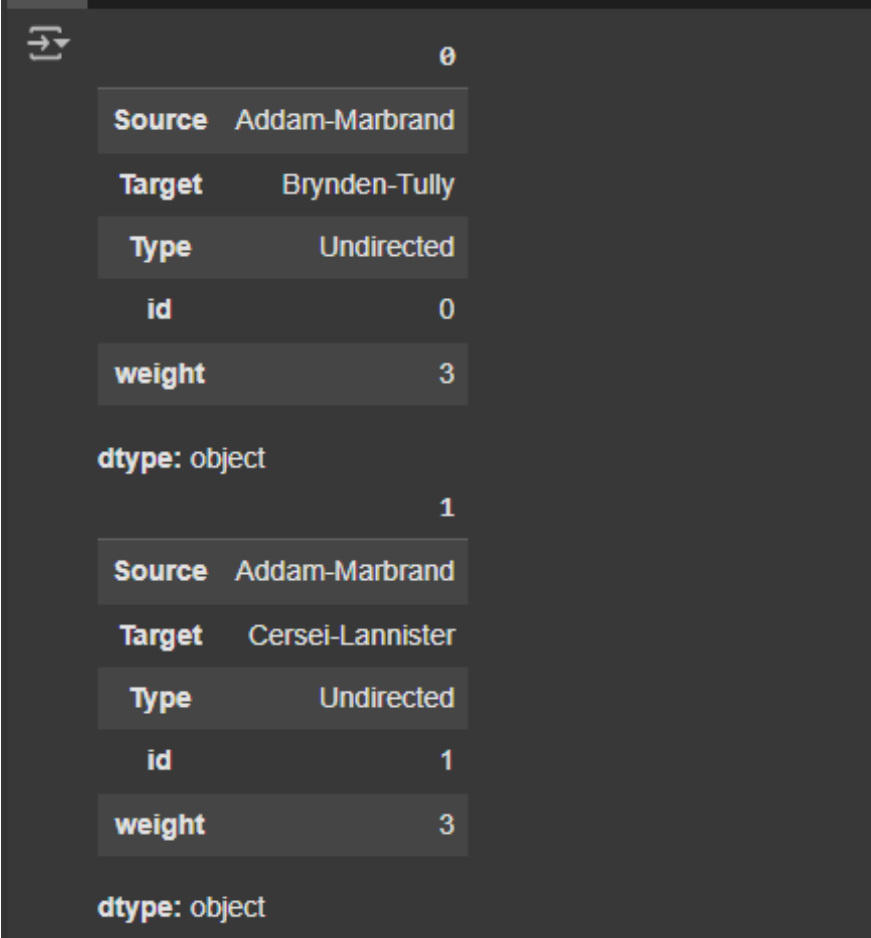
```
rows = edges_df.loc[edges_df["Source"]=="Robb-Stark"]
display(rows["Target"].head(5))
display(rows["Target"].values)
```

	Target
2603	Robert-Baratheon
2604	Robin-Flint
2605	Rodrik-Cassel
2606	Rollam-Westerling
2607	Rolph-Spicer

```
dtype: object
array(['Robert-Baratheon', 'Robin-Flint', 'Rodrik-Cassel',
      'Rollam-Westerling', 'Rolph-Spicer', 'Roose-Bolton', 'Roslin-Frey',
      'Ryman-Frey', 'Samwell-Tarly', 'Sansa-Stark', 'Stafford-Lannister',
      'Stannis-Baratheon', 'Stevron-Frey', 'Stiv', 'Sybell-Spicer',
      'Theon-Greyjoy', 'Torrhen-Karstark', 'Tristifer-IV-Mudd',
      'Tyrion-Lannister', 'Tywin-Lannister', 'Utherydes-Wayn', 'Varys',
      'Walder-Frey', 'Walder-Rivers', 'Wendel-Manderly',
      'Willem-Lannister', 'Wyman-Manderly', 'Yoren'], dtype=object)
```

- **.iterrows()** : iterate through each row of a Dataframe
- It returns each row as a pair (index, row object)
- To see the details of the first and second row (or any row we want to explore)

```
for index, row in edges_df.head(2).iterrows():  
    display(row)
```



Index	Source	Target	Type	id	weight
0	Addam-Marbrand	Brynden-Tully	Undirected	0	3
1	Addam-Marbrand	Cersei-Lannister	Undirected	1	3

Graph Creation

- **Adding the nodes:** For each row of the dataframe **nodes_df** we add a **node** to the graph using the column «**ID**», the column «**Label**» is added as an attribute of the node (character's name)
- **Adding the edges:** For each row of the dataframe **edges_df**, we create an edge between two characters based on the «Source» and «Target» columns, and the weight of the edge is the value of the column «**Weight**», and the id is an attribute of the edge (unique identifier)

```
game_of_graphs = nx.Graph()
for index, row in nodes_df.iterrows():
    game_of_graphs.add_node(row["Id"], label=row['Label'])
for index, row in edges_df.iterrows():
    game_of_graphs.add_edge(row["Source"], row["Target"], weight=row['weight'], id=row["id"])
```

Save the graph

- **pickle.dump** function is used to save the graph as a pickle file in wb(write binary) mode
- This allows us to reuse the graph later without needing to reconstruct it.

```
[22] # Save the graph using pickle
      with open("/content/drive/MyDrive/networks_material/game_of_graphs.pkl", "wb") as f:

          pickle.dump(game_of_graphs, f)
          print("Graph saved successfully!")
```

⇒ Graph saved successfully!

Load the graph

- **pickle.load()** to load the graph from the pickle file (correct path)
- Examining the number of nodes/edges/adjacency nodes of a node n



```
# Load the graph from the pickle file
with open("/content/drive/MyDrive/networks_material/game_of_graphs.pkl", "rb") as f:
    loaded_graph = pickle.load(f)

print("Graph loaded successfully!")
print("Number of nodes:", loaded_graph.number_of_nodes())
print("Number of edges:", loaded_graph.number_of_edges())
print(dict(game_of_graphs.adj["Robb-Stark"]))
```



```
Graph loaded successfully!
Number of nodes: 796
Number of edges: 2823
{'Aegon-Frey-(son-of-Stevron)': {'weight': 5, 'id': 13}, 'Arya-Stark': {'weight': 44, 'id': 348}, 'Balon-Greyjoy': {'
```

Drawing the graph

- **Set Plot Size:** Define the figure size for a clear and organized layout.
- **Load the Graph:** Use the pickle file to load the graph into memory.
- **Spring Layout** with colors of nodes, width of edges, colors of edges, with_labels=True to show the labels for nodes
- **Save** the plot as a pdf and display it

```
import matplotlib.pyplot as plt
from pylab import rcParams
import pickle

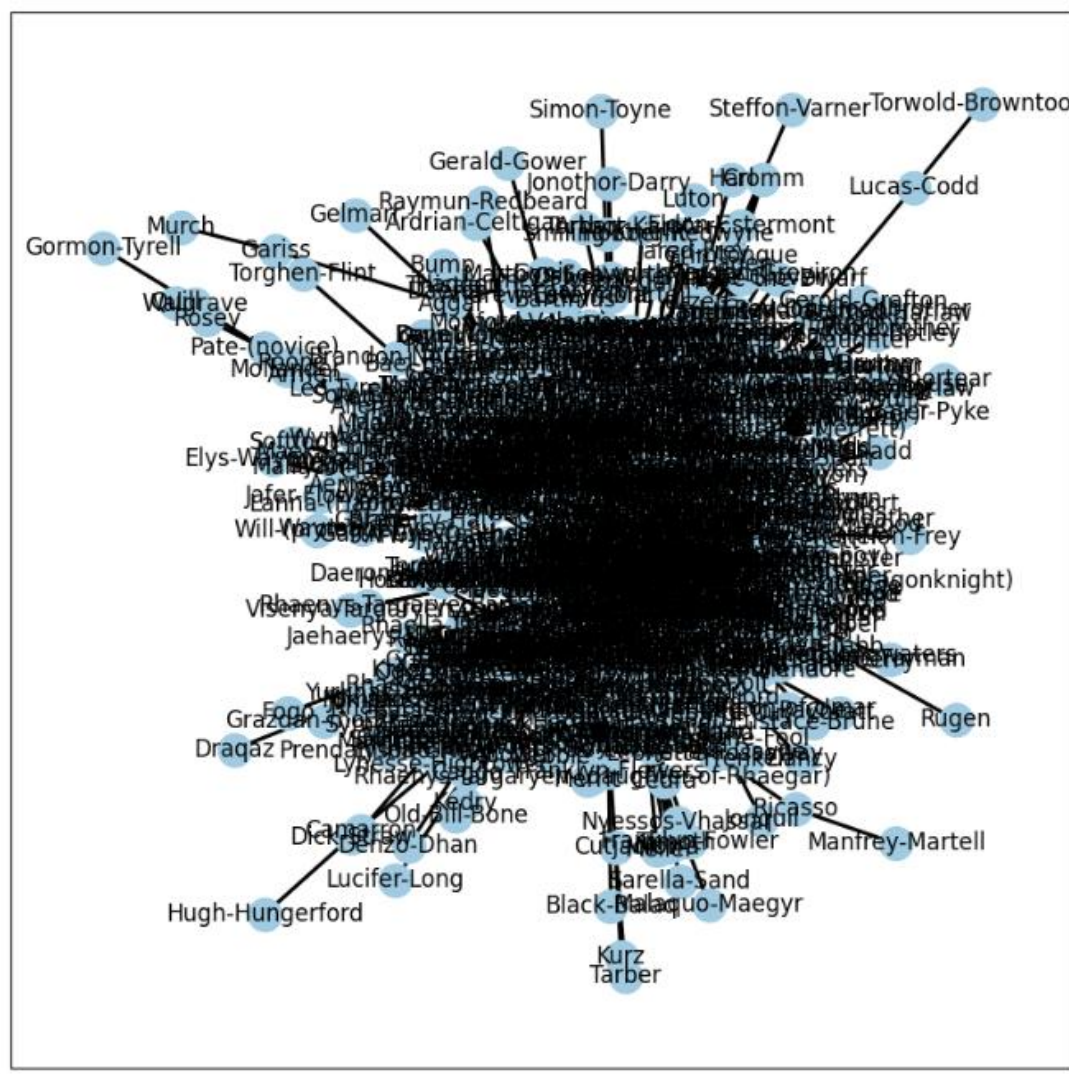
# Set the figure size for the plot
rcParams['figure.figsize'] = 10, 10

# Load the graph from the pickle file
with open("/content/drive/MyDrive/networks_material/game_of_graphs.pkl", "rb") as f:
    game_of_graphs = pickle.load(f)

# Draw the graph using NetworkX's drawing function
nx.draw_networkx(
    game_of_graphs,
    pos=nx.spring_layout(game_of_graphs), # Use a spring layout
    node_color='#A0CBE2',                # Set node color
    width=2,                             # Set edge width
    edge_cmap=plt.cm.Blues,               # Set edge colormap
    with_labels=True                      # Include labels for nodes
)

# Save the plot as a PDF file
plt.savefig("/content/drive/MyDrive/networks_material/game_of_graphs.pdf")
plt.show()
plt.close()
```

Drawing the graph



Reduced graph

Creating a Reduced Graph (Ego Graph):

- Uses the `nx.ego_graph` function to extract a subgraph around a specific node.
- **Ego Graph:** Focuses on a single node and its neighbors within a specified distance.
- **Node:** "Brynden-Tully", **Radius:** 1 (includes immediate neighbors).

Key Steps:

- 1. Load the Graph:** Load the main graph from the pickle file.
- 2. Extract Subgraph:** Generate an ego graph centered on "Brynden-Tully".
- 3. Statistics:** Print the number of nodes and edges in the reduced graph.

```
#Reduced graph
import matplotlib.pyplot as plt
from pylab import rcParams
import pickle

# Set the figure size for the plot
rcParams['figure.figsize'] = 10, 10

# Load the main graph from the pickle file
with open("/content/drive/MyDrive/networks_material/game_of_graphs.pkl", "rb") as f:
    game_of_graphs = pickle.load(f)

# Create a reduced graph (ego graph) around "Brynden-Tully" with a radius of 1
reduced_graph = nx.ego_graph(game_of_graphs, "Brynden-Tully", radius=1)

# Print the number of nodes and edges in the reduced graph
print("Number of nodes in the reduced graph:", reduced_graph.number_of_nodes())
print("Number of edges in the reduced graph:", reduced_graph.number_of_edges())

# Draw the reduced graph
nx.draw_networkx(
    reduced_graph,
    pos=nx.spring_layout(reduced_graph), # Use a spring layout for positioning
    node_color='#A0CBE2',                # Set the color of the nodes
    edge_color="#a9a9a9",                # Set the color of the edges
    width=2,                             # Set the edge width
    with_labels=True                     # Include labels for nodes
)

# Save the visualization as a PDF file
plt.savefig("/content/drive/MyDrive/networks_material/brynden_tully_graph.pdf")
plt.show()
plt.close()
```

Reduced graph

